EC513 · SPRING 2026 · FINAL PROJECT

YAGS

Implementing the Yet Another Global Scheme branch predictor in gem5

A. N. Eden and T. Mudge, “The YAGS branch prediction scheme,” in *Proceedings. 31st Annual ACM/IEEE International Symposium on Microarchitecture*, Dec. 1998, pp. 69–77. doi: 10.1109/MICRO.1998.742770.

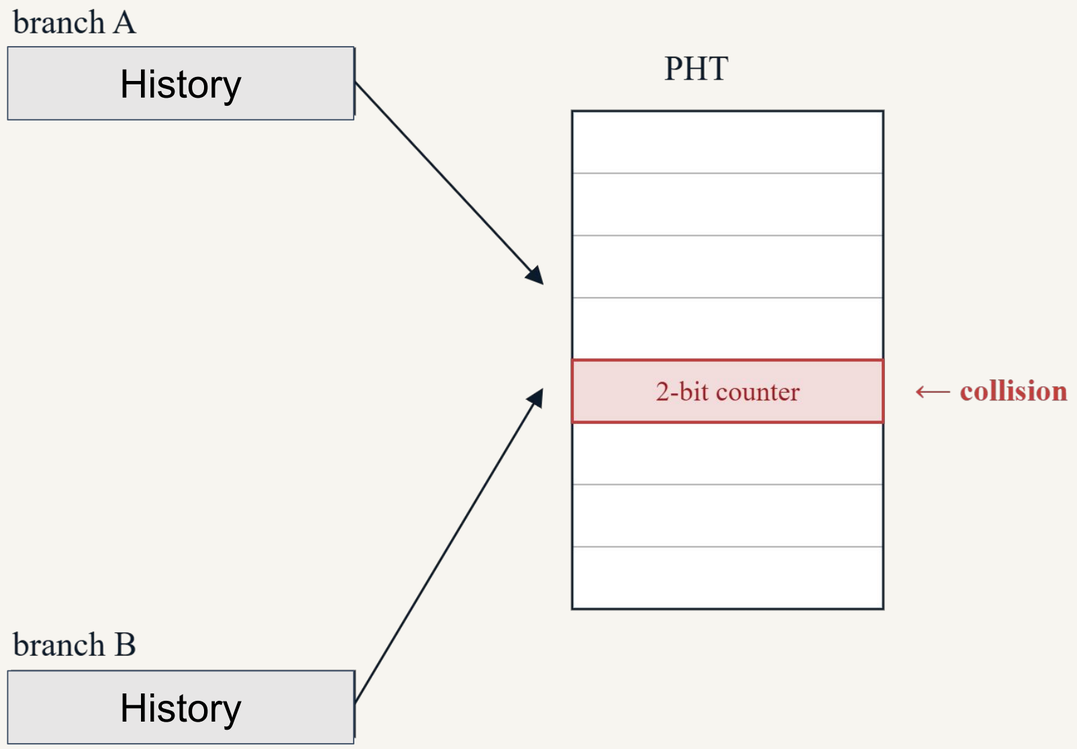
TEAM

Brian Quijada · Fadi Kidess · Ozan Ekame Pekgoz · Will Borland

EDEN & MUDGE · 1998 · UNIV. OF MICHIGAN

Main issue for Global Branch Predictors: PHT Aliasing

- Two different indices map to the same Pattern History Table (PHT) entry
- Aliasing can be neutral:
  - aliased entries share same information
- or destructive:
  - aliased entries disagree and cause a misprediction



Background (1)

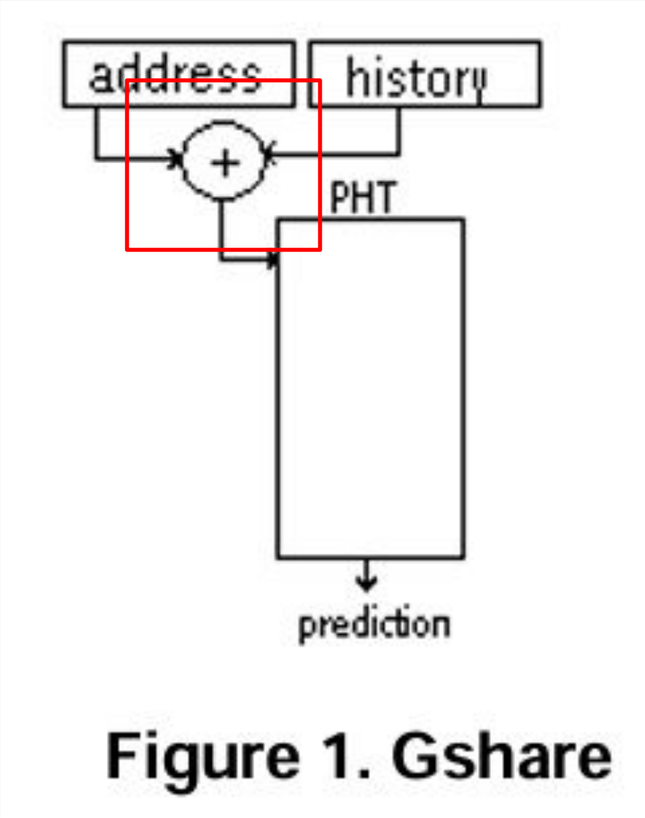


Figure 1. Gshare

- Indexing into the PHT using XOR of branch address and global history
- provides a more even use of PHT entries.

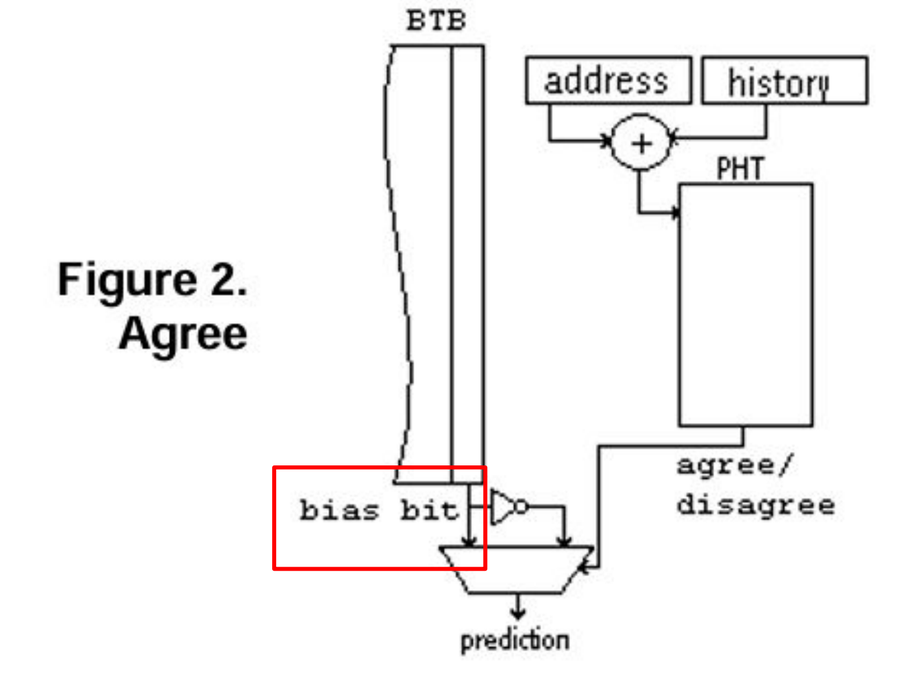


Figure 2. Agree

- Adds a biasing bit to each BTB entry based on direction before it's written
- Main idea: branches tend to be biased towards T or NT
  - agree predictor hopes the first time a branch sees BTB it will take or not take according to that bias
- PHT then goes from T/NT to agree/disagree
- if most branches are biased, most PHT entries will be "agree" -> more neutral, less destructive aliasing

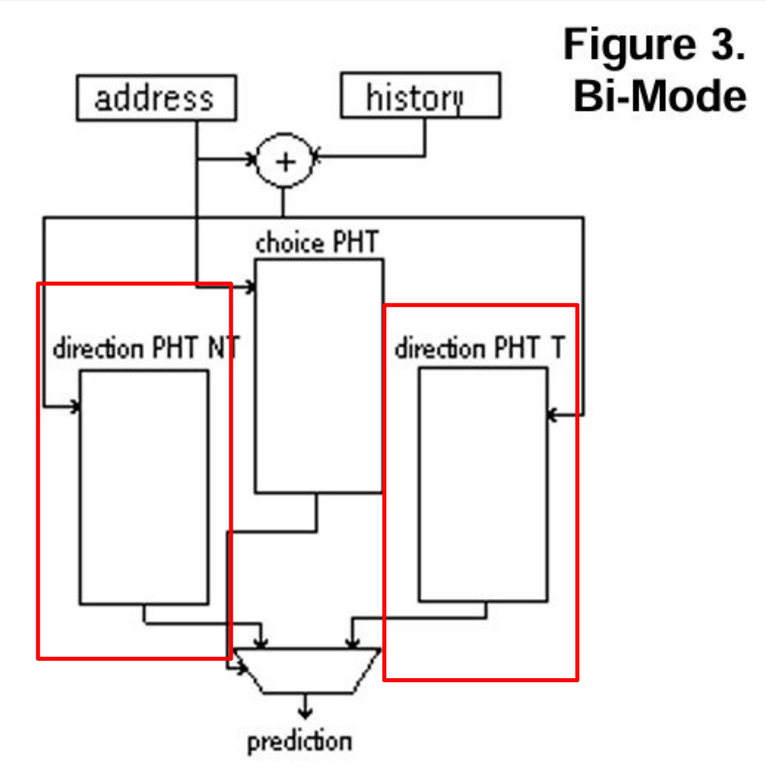
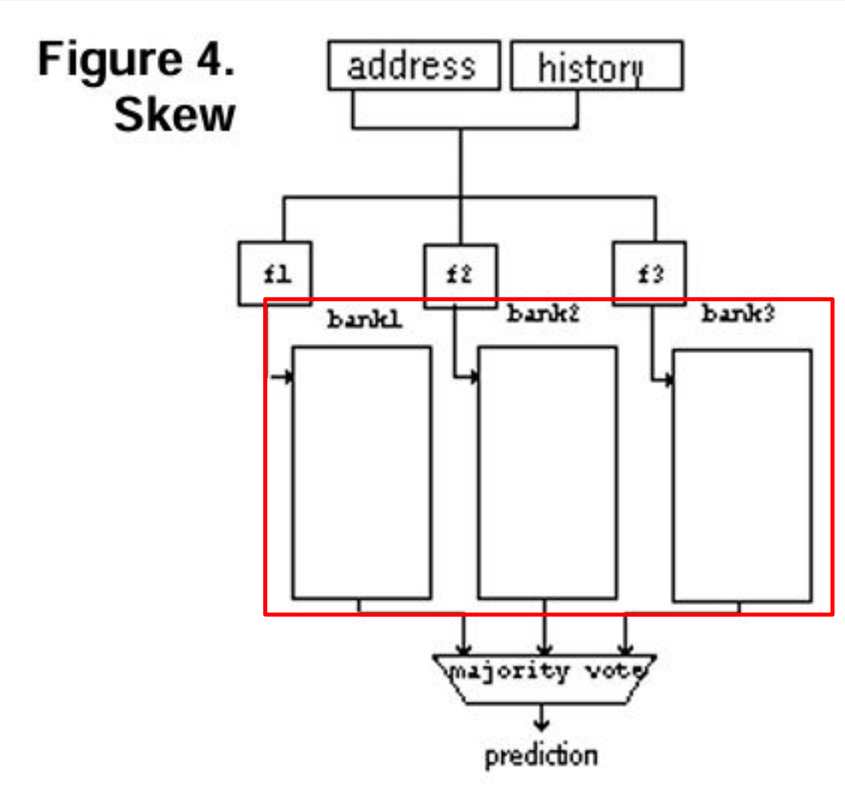


Figure 3. Bi-Mode

- splits PHT:
  - choice PHT: array of Sat. counters
  - directional PHTs: store branches biased in their direction
  - flow : BTB -> XOR -> choice PHT -> choose between T/NT directional PHT -> prediction
  - bias not "locked" like in agree

Background (2)



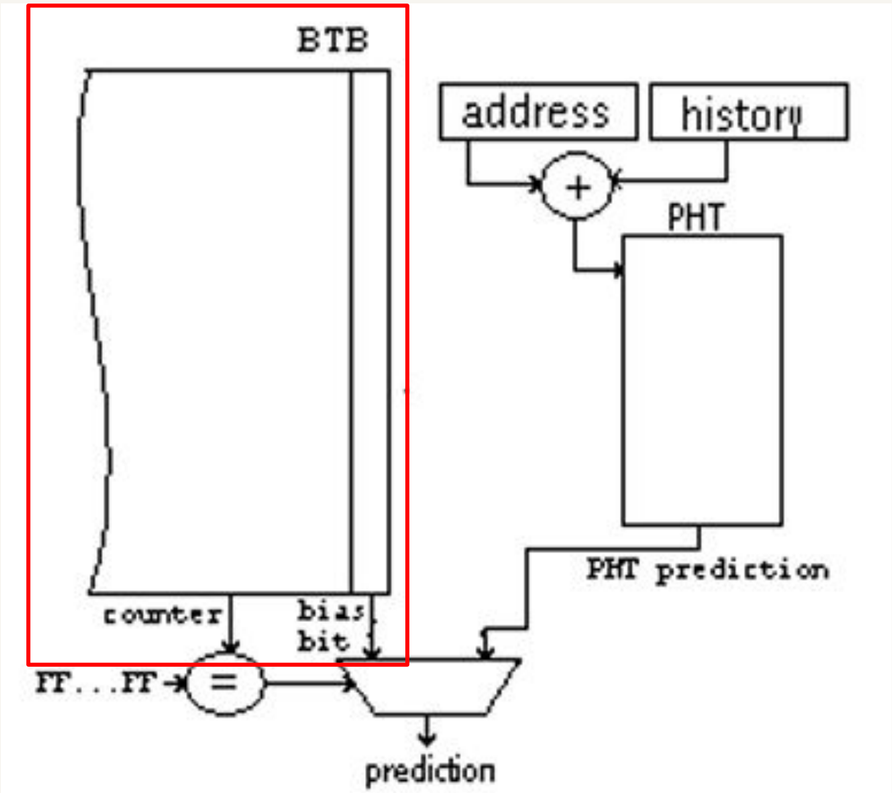


Figure 5. Filter

- **most aliasing comes from lack of associativity**, not lack of size
- PHT split into **3 banks**: each has a unique hashing function
- 3 banks **'vote'** -> prediction
- unlike previous: tries to eliminate aliasing between a branch that has **T bias colliding with NT instances of that branch**
- **tradeoff**: capacity aliasing for conflict aliasing

- goal is to **reduce redundancy** inside PHT
- **filters out highly biased branches** in BTB
- PHT only used if branch counter NOT saturated
- tries to reduce all aliasing by reducing total information in PHT

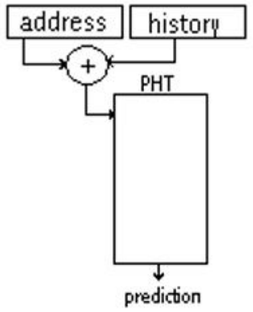


Figure 1. Gshare

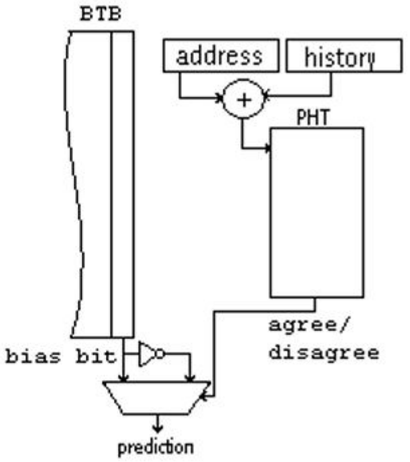


Figure 2. Agree

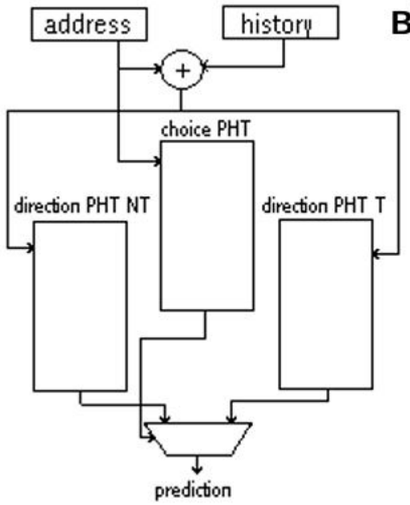


Figure 3. Bi-Mode

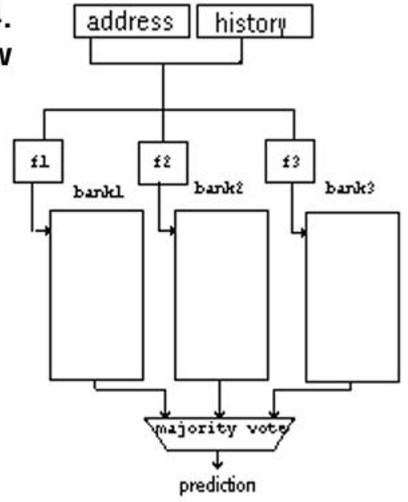


Figure 4. Skew

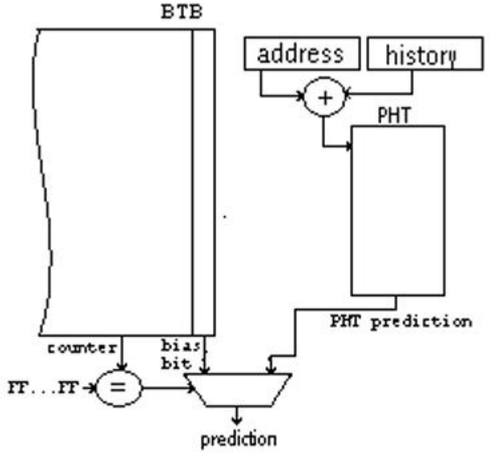


Figure 5. Filter

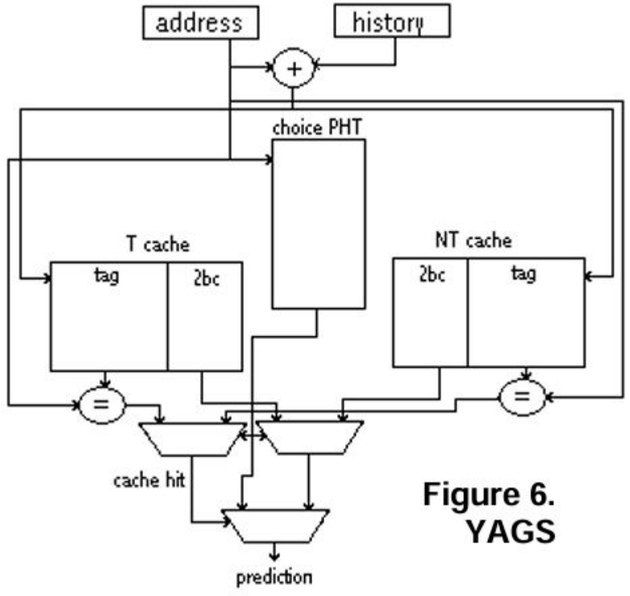


Figure 6. YAGS

mostly taken straight from paper

yags

- splits branches into NT and T ‘streams’ (Bimode)
- wants to account for aliasing between
- “aliasing between biased branches and their instances which do not comply with that bias”
  - that is: When a branch that usually goes one way, occasionally goes the other way (skewed)
- reduce unnecessary information in PHT (filter) without additional mispredictions

- YAGS wants to store each branches bias as well as the instances that don’t agree with that bias
- bimodal (sat counters) to store bias - “choice predictor”
- reduces information stored in direction PHTs
  - so much so that they are smaller than choice

- add small tags to entries in direction PHTs
  - now called “direction caches” (fancy)
  - tags store LSB of branch address, nearly eliminates aliasing between two consecutive branches

- flow: Branch Instruction seen -> PHT accessed -> if (ex.) PHT indicates taken -> NT cache is accessed to check if it is a ‘special case’ (pred doesnt agree w/ bias) (if NT $ miss, choice PHT used as prediction) (if NT $ hit, it supplies prediction)

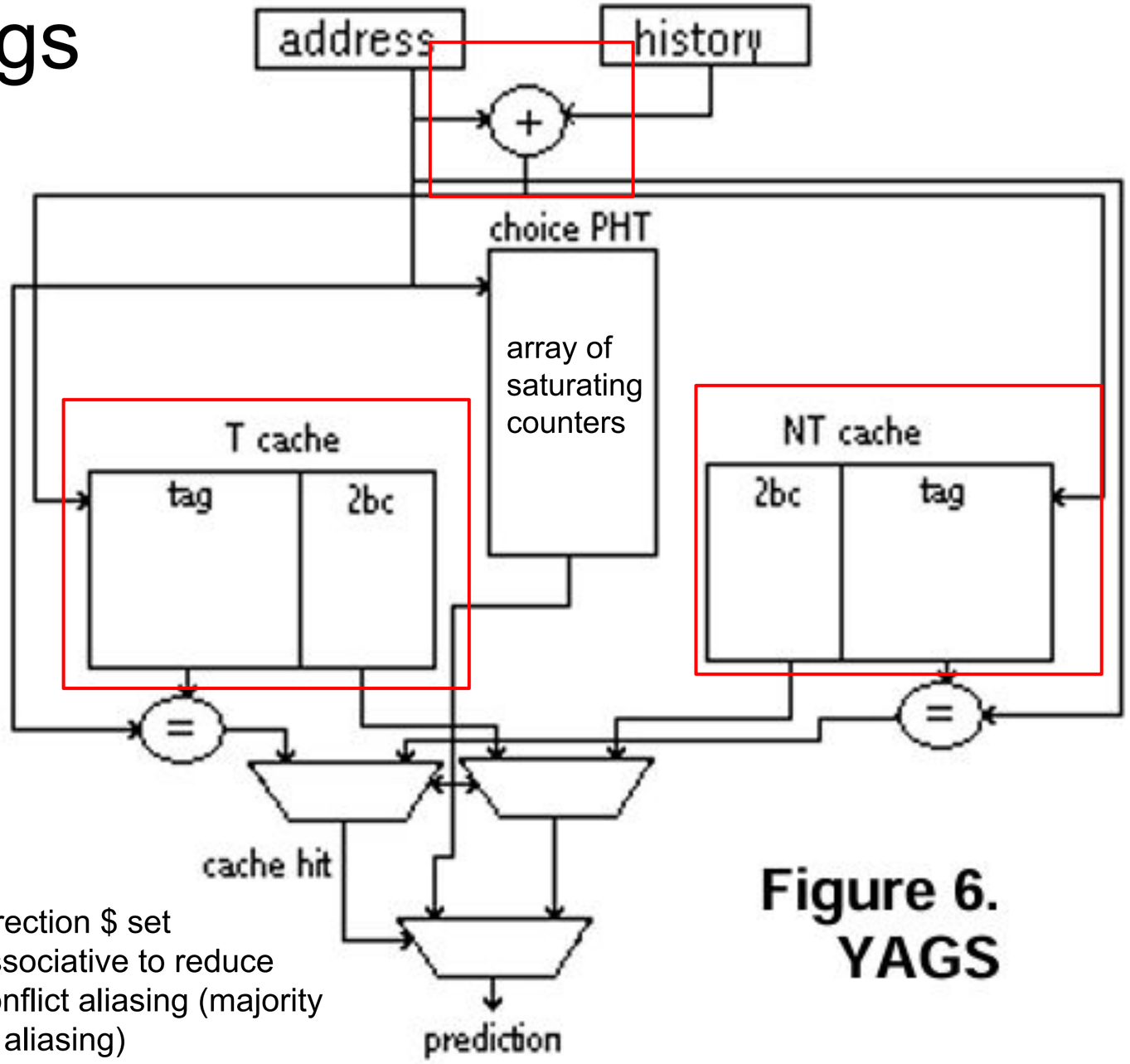


Figure 6. YAGS

- direction $ set associative to reduce conflict aliasing (majority of aliasing)
- basically LRU replacement + an entry in the “taken” cache which indicates “not taken” will be replaced first to avoid redundant information.

YAGS combines ideas from all of these

- address XOR history used to index T/NT Cache (Gshare)
- directional caches for Taken or Not Taken (bimode)
- does not ignore aliasing between biased branches and instances which don't align with its bias (agree / skewed)
- aims to reduce unnecessary information in PHT (filter)

- on the high level, YAGS wants to **store each branch's bias** (choice PHT) and the **instances that don't agree** with that bias (T/NT directional PHT)
  - less information in T/NT PHT -> add 6-8 bit tags to directional PHTs and **call it a Cache**
  - set associative directional caches reduces conflict aliasing (largest portion of aliasing in PHT)
  - LRU replacement, with exception that any T$ entries indicating NT are removed first (or vice versa)
- on **T/NT $ miss**, use choice PHT prediction

Example Flow: Branch biased towards T, here it goes NT
(again, as the first time would create entry in NT $)

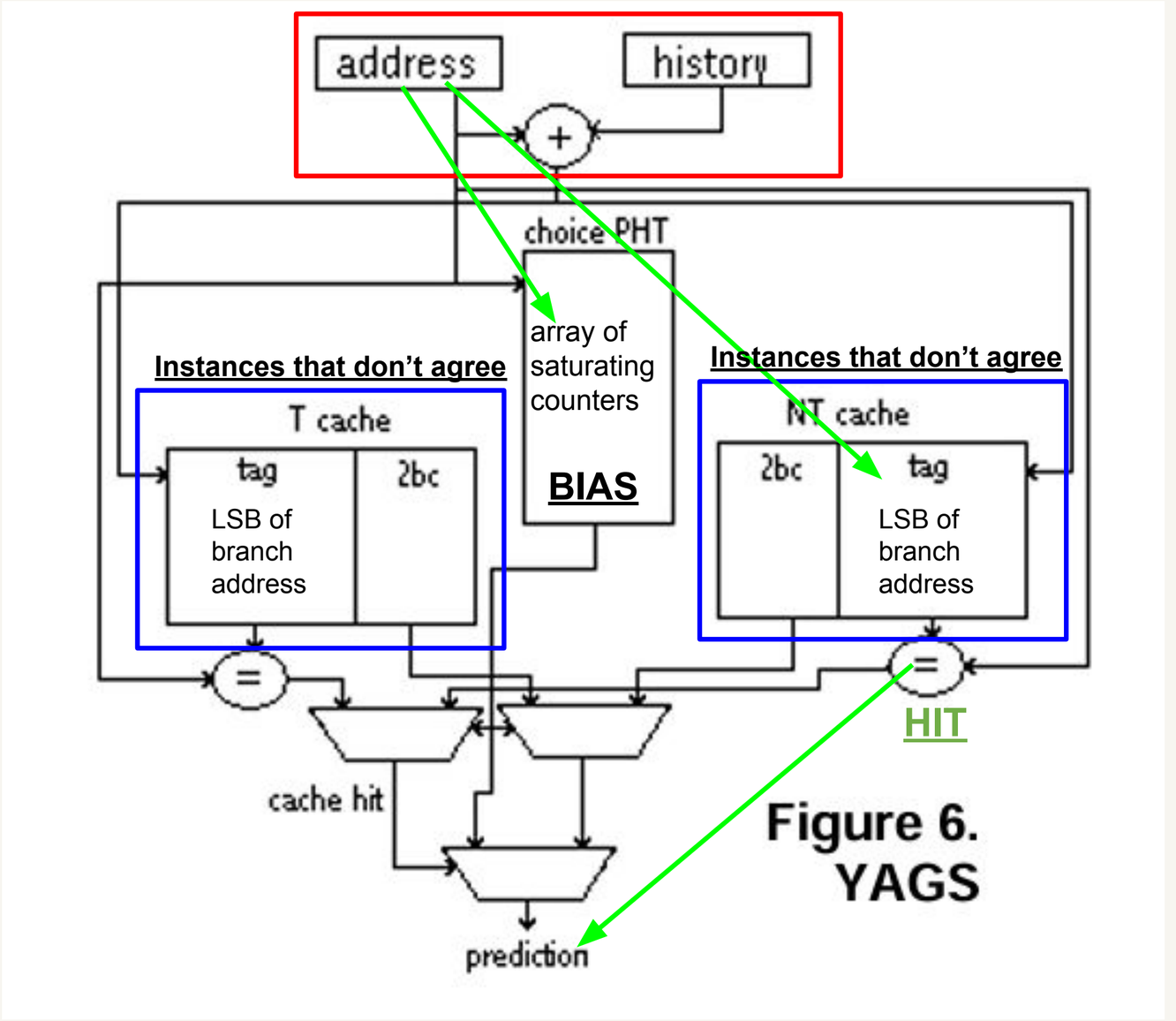


Figure 6. YAGS

YAGS stores only the exceptions to a branch's bias and tags them.

A bimodal **choice PHT** records each branch's bias. Two small **tagged direction caches** (one for "taken" exceptions, one for "not taken") hold only the cases that disagree with the bias.

01	Choice PHT predicts taken → look up the *not-taken* cache.
02	Tag hit → use the cache. Miss → fall back to the choice.
03	6–8 bit tags virtually eliminate aliasing between consecutive branches.

Direction caches can be ½, ¼, even ⅛ the size of the choice PHT. Tags are paid for by the entries we no longer need.

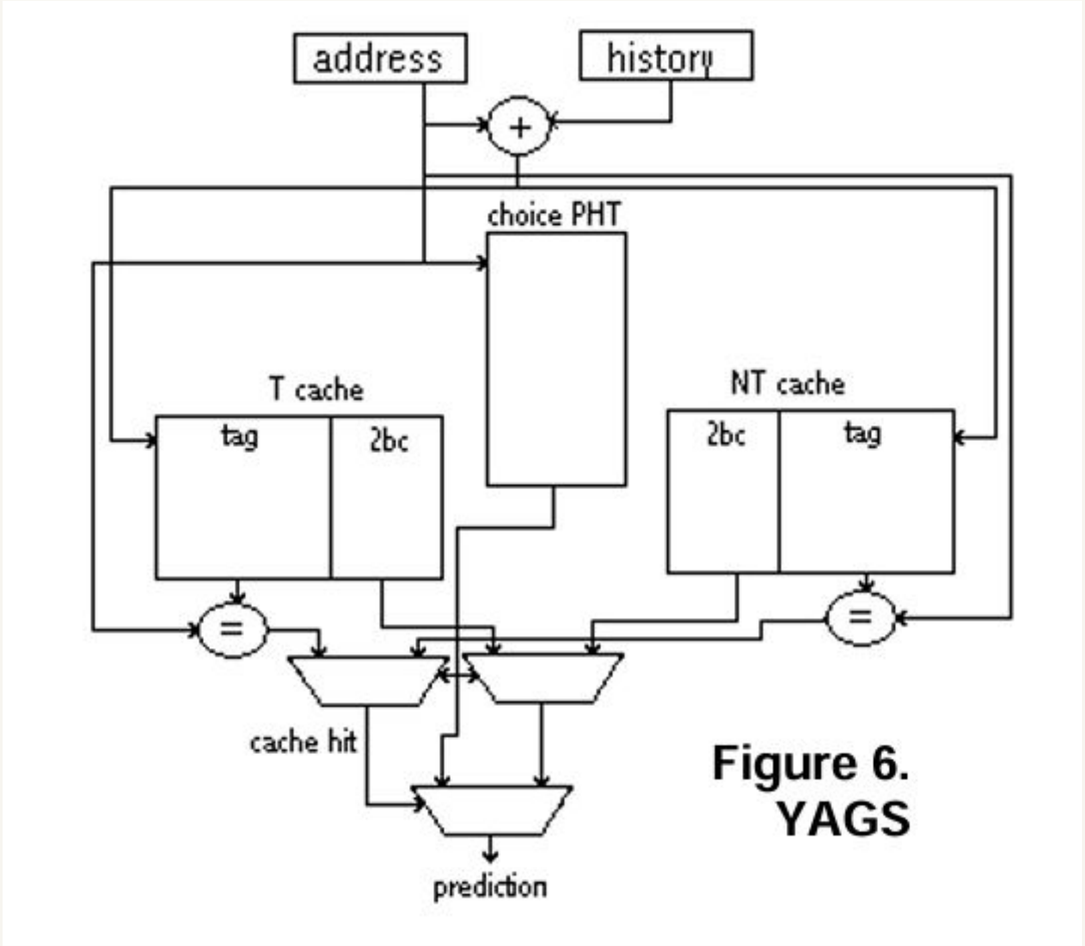


Figure 6. YAGS

How we implemented a new BPredUnit subclass alongside the existing predictors.

SOURCE TREE

```
gem5/src/cpu/pred/
├─ bpred_unit.{hh,cc}        (base class)
├─ bi_mode.{hh,cc}  ├─ tournament.{hh,cc}  ├─ yags.hh  ├─ yags.cc  └─ BranchPredictor.py        # SimObject
```

DATA STRUCTURES

choicePHT[ ]	SatCounter8 (2-bit) · indexed by PC
takenCache[ ]	{tag, 2-bit ctr, valid} · PC ⊕ history
notTakenCache[ ]	{tag, 2-bit ctr, valid} · PC ⊕ history

CONFIGURATION

choicePHTSize	4096
cacheSize (each)	2048
tagBits	8
globalHistoryBits	11
ctrBits	2

LOOKUP

```
choice = choicePHT[PC];
if (choice predicts T) {
  if (NT_cache.hit(PC, hist))
    return NT_cache.pred;
// exception    return T;                    // fallback } else { ... symmetric ... }
```

How we implemented a new BPredUnit subclass alongside the existing predictors.

Choice PHT was implemented as a vector of saturating counters. The address and history

The caches were also implemented as vectors of saturating counters, with the tags being unsigned vectors containing the lower MSB of the address XORed with the history.

Both caches and tags had the same depth of 8192 for consistency over the benchmarks.

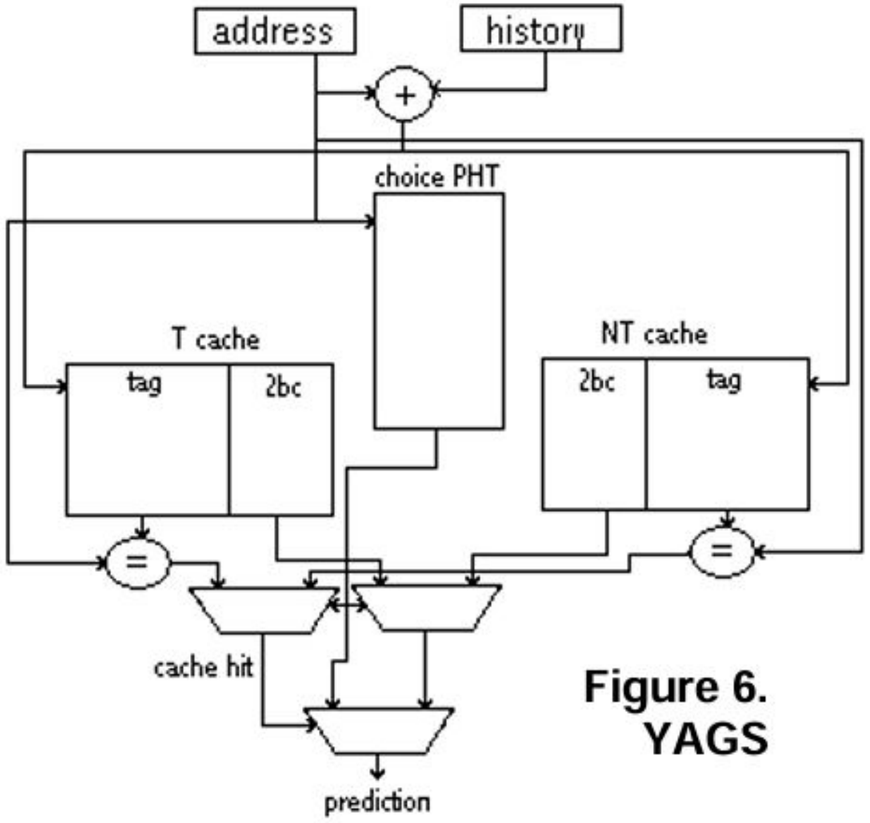


Figure 6.
YAGS

Problems we faced and our solutions

ISSUE	WHAT BROKE	SOLUTION
Tag & index sizing	A segmentation flow occurred initially as we didn’t implement modulo indexing, so the tag address overflowed.	We implemented modulo indexing and also increased the cache size.
Long sim turnaround	5B-warmup + 1B-measure across 5 benchmarks took 4/5 days - full-run debugging was infeasible.	We simulated only with a 1B instruction-warmup due to time constraints.

Benchmarks & Simulation methodology

502 - gcc_r	505 - mcf_r	520 - omnet_pp	523 - xalancbmk_r	525 - x264_r
C compiler	Route planning - Lots of integer arithmetic	more unpredictable network simulation	XML to HTML conversion - more predictable	Video compression

Methodology:

- Attempted 5 billion instruction warmup, 1 billion instructions measured
  - Simulations took too long → used 1 billion/1 billion instead
  - This was a problem (seen in results)
- Compared YAGS to Gshare, Tournament, Tage, Local, and MultiperspectivePerceptron 8KB
  - IPC, misprediction rate

Five SPEC CPU 2017 benchmarks against the three HW5 predictors.

BENCHMARKS

505	mcf_r
520	omnetpp_r
523	xalancbmk_r
502	gcc_r
525	x264_r

METHODOLOGY

1B instructions warm-up
predictors reach steady state

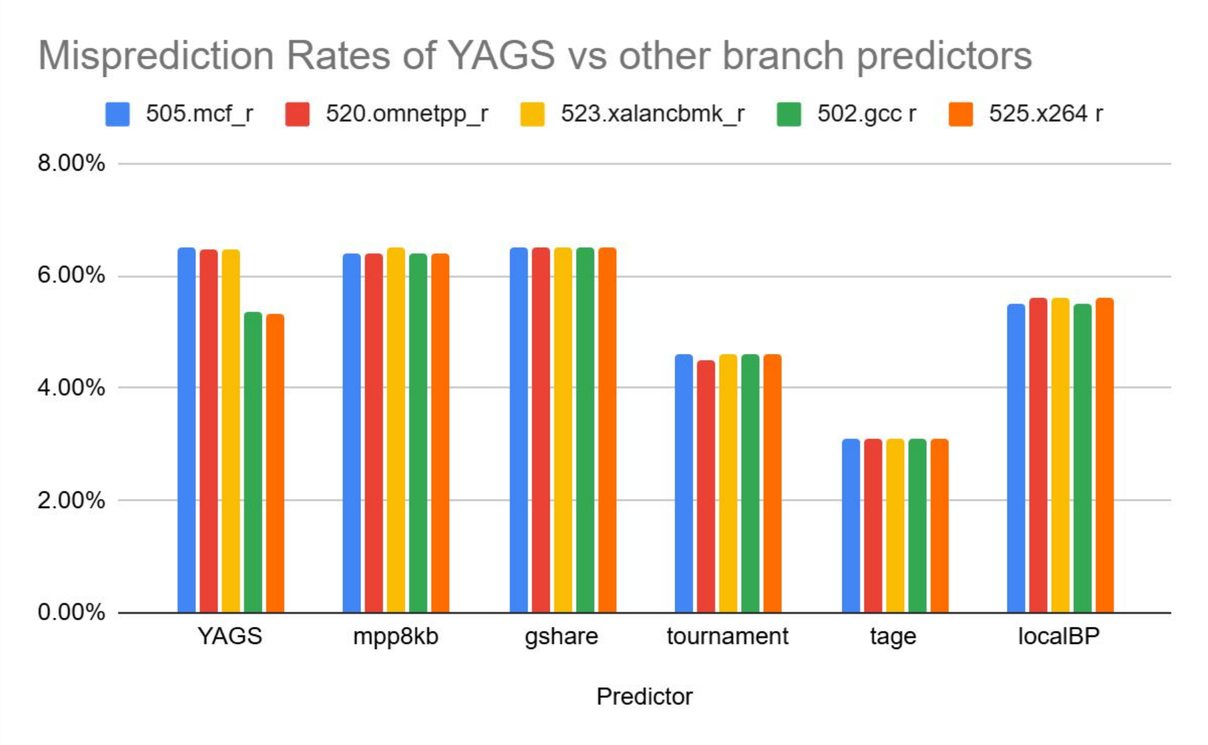
1B instructions measured
IPC + misprediction rate from m5out

O3CPU baseline config
se.py · same memory hierarchy across runs

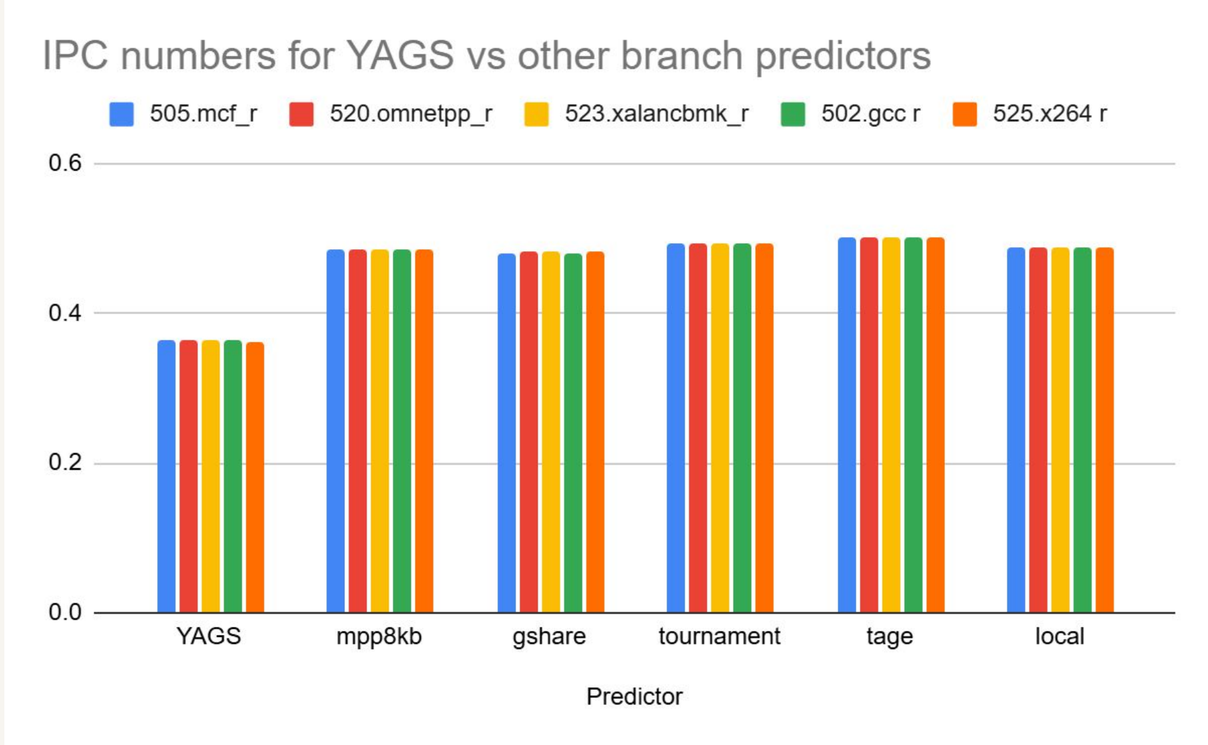
vs. HW5 predictors
TournamentBP · LTAGE · MPP-TAGE 64KB

Simulation Results

MISPREDICTION RATE (%) — LOWER IS BETTER



IPC — HIGHER IS BETTER



What went wrong with our simulations:

Branch Misprediction Rates

Predictor	505.mcf_r	520.omnetpp_r	523.xalancbmk_r	502.gcc r	525.x264 r
YAGS	**6.5%**	**6.47%**	**6.46%**	**5.35%**	**5.33%**
mpp8kb	6.4%	6.4%	6.5%	6.4%	6.4%
gshare	6.5%	6.5%	6.5%	6.5%	6.5%
tournament	4.6%	4.5%	4.6%	4.6%	4.6%
tage	3.1%	3.1%	3.1%	3.1%	3.1%
localBP	5.5%	5.6%	5.6%	5.5%	5.6%

IPC

Predictor	505.mcf_r	520.omnetpp_r	523.xalancbmk_r	502.gcc r	525.x264 r
YAGS	**0.363**	**0.363**	**0.363**	**0.363**	**0.361**
mpp8kb	0.484	0.484	0.484	0.484	0.484
gshare	0.481	0.482	0.482	0.481	0.482
tournament	0.493	0.493	0.492	0.492	0.492
tage	0.501	0.501	0.501	0.501	0.501
local	0.488	0.489	0.489	0.488	0.489

If we had to redo, what we'd tell next semester's class

IF WE WERE ABLE TO RESTART

- Sweep tag size and direction-cache ratio. To do so, make sure all variables are parameterizable.
- Set up experiment batch jobs no less than two weeks in advance. 5 billion warmup instruction simulations took much longer than we expected, and 1 billion is insufficient in warmup.

FOR NEXT SEMESTER

- Ensure simulation setup is valid to prevent future reruns of experiments.
- Parameterize your solution to allow for flexible analysis of different variations of your design.

Thank you! / Questions?

All images / diagrams:

A. N. Eden and T. Mudge, “The YAGS branch prediction scheme,” in *Proceedings. 31st Annual ACM/IEEE International Symposium on Microarchitecture*, Dec. 1998, pp. 69–77. doi: 10.1109/MICRO.1998.742770.

TAKEAWAYS

Tags, smaller direction caches, and a bimodal choice: three small ideas, one solid predictor.

~1 KB

predictor footprint at our chosen config

5 / 5

SPEC2017 benchmarks evaluated against HW5

8 bit

tags — past which paper shows diminishing returns

QUESTIONS

Thank you.

REFERENCE A. N. Eden & T. Mudge — *The YAGS Branch Prediction Scheme* MICRO-31, 1998 · Univ. of Michigan EECS